

Processador RISC-V RV32I com Pipeline de 5 Estágios e Extensão Vetorial (SIMD)

Artur Taboada, Pedro Igor Pereira e Kauã Alves

Arquitetura de Computadores — UFRJ

Resumo

Este trabalho apresenta o projeto de um processador RISC-V de 32 bits (subconjunto inteiro RV32I) com pipeline de cinco estágios, memórias de instrução e dados separadas com carga assíncrona, tratamento completo de riscos (*hazards*) por encaminhamento e congelamento, e uma extensão vetorial (SIMD) de 128 bits com quatro *lanes* de 32 bits operando em paralelo ao caminho escalar. A lógica foi validada por um modelo ciclo-a-ciclo que reproduz a microarquitetura, com resultados coincidentes aos esperados em todos os testes.

1 Visão geral

O processador implementa o pipeline clássico de cinco estágios: **IF** (busca da instrução), **ID** (decodificação e leitura de registradores), **EX** (execução na ALU), **MEM** (acesso à memória de dados) e **WB** (escrita no banco). As memórias de instruções (ROM) e de dados (RAM) são separadas e entregam uma palavra por ciclo. Há **reset** assíncrono e uma entrada **load_enable** que congela todo o estado interno para permitir a carga assíncrona das memórias.

A implementação foi feita em VHDL, com cada bloco funcional isolado em seu próprio módulo e exposto ao simulador Digital. Blocos combinacionais de múltiplos bits pré-prontos do Digital foram evitados; em particular, a ALU foi descrita explicitamente em VHDL.

2 Conjunto de instruções escalar

São suportadas as instruções: **add**, **addi**, **auipc**, **sub**, **and**, **andi**, **or**, **ori**, **xor**, **xori**, **sll**, **slli**, **srl**, **srli**, **lw**, **lui**, **sw**, **jal**, **jalr**, **beq** e **bne**.

A operação da ALU é resolvida no estágio ID, pela unidade de controle, e propagada como um código de 4 bits (**ALUCtrl**) pelo registrador ID/EX. A Tabela 1 resume a codificação.

ALUCtrl	Operação
0000	ADD
0001	SUB
0010	AND
0011	OR
0100	XOR
0101	SLL
0110	SRL

Tabela 1: Codificação da operação da ALU.

A instrução `sub` é distinguida de `add` pelo campo `funct7` (0100000). `lui` força o operando A a zero (resultado igual ao imediato); `auipc` usa o PC como operando A; `jal` e `jalr` gravam o endereço de retorno (PC+4) no registrador destino.

3 Riscos (*hazards*) e sua resolução

3.1 Encaminhamento (*forwarding*)

Uma unidade de encaminhamento compara o destino das instruções nos estágios EX/MEM e MEM/WB com as fontes da instrução em EX e injeta o valor mais recente nos operandos da ALU (códigos 10 para EX/MEM e 01 para MEM/WB).

3.2 Banco de registradores com *write-through*

O encaminhamento EX/MEM e MEM/WB cobre dependências de distância 1 e 2. A dependência de distância 3 — a instrução produtora escreve no WB no mesmo ciclo em que a consumidora lê no ID — foi fechada dando ao banco de registradores leitura com *bypass* de escrita: se, no mesmo ciclo, se escreve em `rd` e se lê o mesmo endereço, a leitura já devolve o dado novo. Isso evita um estágio extra de bolha.

Listagem 1: Leitura com *write-through* no banco de registradores.

```
dout1 <= din when (we = '1' and rd = rs1) else
    registers(to_integer(unsigned(rs1)));
dout2 <= din when (we = '1' and rd = rs2) else
    registers(to_integer(unsigned(rs2)));
```

3.3 Bolha de *load-use*

Como `lw` só disponibiliza o dado ao fim do estágio MEM, uma instrução dependente imediatamente seguinte exige uma bolha. A unidade de detecção de *hazard* identifica o caso (`lw` em EX cujo destino coincide com uma fonte em ID), congela o PC e o registrador IF/ID e zera os controles da instrução seguinte por um ciclo.

3.4 Desvios e saltos

As instruções `beq/bne` e `jal/jalr` são resolvidas no estágio EX. Quando o desvio é tomado (ou é salto incondicional), as duas instruções buscadas indevidamente são descartadas (*flush* de IF/ID e ID/EX) e o PC é redirecionado. O alvo é $PC + imm$ para `beq/bne/jal` e $(rs1 + imm) \& \sim 1$ para `jalr`.

4 Congelamento para carga assíncrona

Quando `load_enable_i = '1'`, o sinal interno de habilitação de escrita do pipeline (`pipe_we`) vai a '0'. Com isso o PC, todos os registradores de pipeline e o banco de registradores param de atualizar: o estado inteiro fica congelado enquanto as memórias são carregadas externamente de forma assíncrona. Ao retornar `load_enable_i = '0'`, a execução prossegue exatamente de onde parou.

5 Extensão vetorial (SIMD)

O caminho vetorial reaproveita integralmente o pipeline escalar e adiciona, em paralelo, um banco vetorial de 32 registradores de 128 bits, uma ALU vetorial de quatro *lanes* de 32 bits,

encaminhamento vetorial e registradores de pipeline vetoriais (ID/EX, EX/MEM e MEM/WB). O banco vetorial também recebeu *write-through*, pela mesma razão do banco escalar.

Uma instrução vetorial é, para o controle escalar, uma instrução de opcode desconhecido e portanto tratada como NOP (não escreve no banco escalar nem acessa memória). Os bancos escalar e vetorial são independentes, o que evita conflitos entre os dois caminhos.

5.1 Codificação das instruções vetoriais

As instruções vetoriais usam **opcodes customizados** do espaço reservado do RISC-V, preservando a aderência ao padrão: os campos **rs1**, **rs2**, **rd**, **funct3** e **funct7** ocupam as mesmas posições do RV32I. A Tabela 2 descreve a codificação.

Instrução	Tipo	opcode	funct3	funct7	Operação (por <i>lane</i>)
vadd	R	1011011	000	0000000	$a + b$
vsub	R	1011011	000	0100000	$a - b$
vsll	R	1011011	001	—	$a \ll (b \bmod 32)$
vsrl	R	1011011	101	—	$a \gg (b \bmod 32)$
vaddi	I	0001011	000	—	$a + imm$ (difundido)
vslli	I	0001011	001	—	$a \ll imm$
vsrli	I	0001011	101	—	$a \gg imm$
vauipc	U	0101011	000	—	$PC + (imm \ll 12)$ (difundido)

Tabela 2: Codificação das instruções vetoriais.

Cada registrador vetorial vN de 128 bits é interpretado como quatro palavras de 32 bits: $vN = [lane_3 \mid lane_2 \mid lane_1 \mid lane_0]$. Nas instruções com imediato, o imediato de 32 bits é **difundido** (*broadcast*) para as quatro *lanes* antes da operação. A ALU vetorial reutiliza a mesma tabela de operações da ALU escalar.

5.2 Limitação e extensão natural

Seguindo o enunciado, a ISA vetorial adotada não inclui **load** e **store** vetoriais. Por isso, no programa de teste, os registradores vetoriais são inicializados por difusão (**vaddi vd, v0, K**), o que faz as quatro *lanes* partirem do mesmo valor; a independência real entre *lanes* é demonstrada no teste unitário da ALU vetorial com *lanes* distintas. Um **vlw/vsw** de 128 bits seria a extensão natural para popular *lanes* com valores diferentes.

6 Metodologia de verificação e resultados

Para validar a lógica, além da simulação no Digital foi desenvolvido um **modelo de referência ciclo-a-ciclo em Python** que reproduz exatamente a microarquitetura (mesmos estágios, mesmo encaminhamento, mesmo *write-through*, mesma resolução de desvio no EX), permitindo conferir automaticamente os valores dos registradores a cada ciclo. Os principais resultados:

- **Cobertura de instruções:** um programa que exercita as 21 instruções exigidas produziu resultados corretos em todos os registradores, incluindo **lui**, **auipc**, **jal** e **jalr** (endereço de retorno e alvo calculado corretos).
- **Programa de teste fornecido:** executado por 90 ciclos, os valores finais coincidiram exatamente com os esperados anotados no arquivo (por exemplo $x1 = 0xA$, $x5 = 0x28$, $x14 = 0x3C$, $x17 = 3$). As bolhas de *load-use* e os *flushes* de desvio ocorreram nos pontos esperados.
- **SIMD:** o teste unitário da ALU vetorial com quatro *lanes* distintas confirmou a operação *lane-a-lane* (por exemplo $[10, 20, 30, 40] + [1, 2, 3, 4] = [11, 22, 33, 44]$); o programa vetorial produziu os resultados esperados em todas as *lanes*.

- **Verificação estrutural:** um verificador de portas confirmou que todas as instâncias dos dois *top-levels* conectam portas existentes, com direção correta e sem portas desconectadas.

7 Organização dos arquivos e submissão

O projeto é entregue em duas partes independentes, cada uma autossuficiente:

- **Parte 1 (escalar):** módulos VHDL do processador, `Circuito.dig`, `programa.hex` e `test_vector.asm`.
- **Parte 2 (SIMD):** processador com a extensão vetorial (inclui cópias dos módulos escalares, pois é uma CPU completa mais o caminho vetorial), `Circuito_SIMD.dig` e o programa de teste vetorial.
- **Ferramentas:** montador e modelos de verificação em Python.

Nas pastas da Parte 2, os módulos `instruction_decoder`, `control_unit` e `register_file` diferem dos da Parte 1 (o decodificador reconhece os imediatos vetoriais), portanto as duas pastas não devem ser misturadas.